

Teaching a One-Million-Parameter Model to Read Dates

Free synthetic data, an honest capability map, and a clean capacity ceiling

A Technical Report by **Vuk Rosić**

Subject: **nano-dates**, an open-source (MIT) 1M-parameter model that converts natural date phrases into ISO-8601.

Weights: huggingface.co/vukrosic/nano-dates · Code: github.com/vukrosic/nano-dates. June 2026.

Abstract. nano-dates is a 1,016,960-parameter byte-level transformer that turns a natural date phrase into an ISO-8601 date — for example, “the 3rd of July 2025” becomes 2025-07-03. It runs on a CPU in milliseconds and trains from scratch in about thirty seconds on one GPU. The point of the project is not the model but the *method*: every training example is produced by code rather than scraped or labelled. Because the task is narrow and formally specified, you can sample the answer first and then render it in many natural surface forms, so the label is correct by construction — free, unlimited, and exact. This report describes the recipe, a data-leak bug that made an earlier model look perfect while it was quietly cheating, and an honest map of exactly where a model this small stops being able to reason. It is a method demonstration and a capability study, not a production date library.

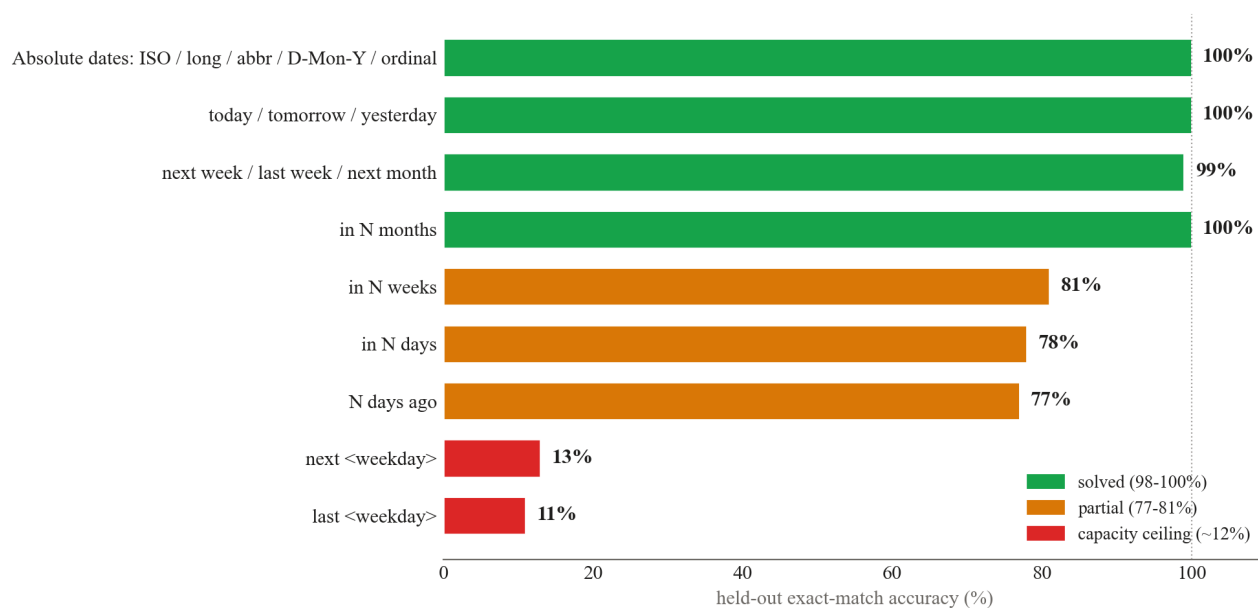


Figure 1. Held-out exact-match accuracy on 2,000 unseen examples (greedy decode), overall **85.4%**. Green = solved; amber = partial (variable-count arithmetic); red = the capacity ceiling. Read the bottom two bars: resolving “next friday” is the one thing the model cannot do, and it is the most computationally entangled item in the set.

1. Why generate the training data with code

A 1M-parameter model cannot be a general assistant, but it can completely nail a task that is narrow and formally specified. Date→ISO is exactly that, and that is what makes the data free. The key move is to **sample the answer first**: pick a date, then emit it as “June 12, 2023”, “Jun 12 2023”, “the 12th of June 2023”, or — for relative phrases — pick a reference date and emit “tomorrow” or “in 3 weeks”. Because the answer is where you started, the label is correct by construction.

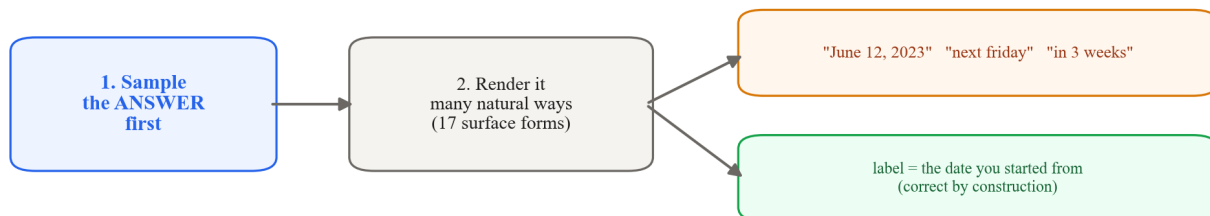


Figure 2. The recipe. Start from the answer date, render it many ways, and the label is simply the date you began with. No verification, no annotation cost, no distillation from a larger model.

This is strictly better than asking a large model to produce training data: there is nothing to verify (the label *is* the ground truth you generated), it costs nothing and is unlimited, and you control the exact distribution of forms and difficulty. The model is handed a reference date (today) at the start of every prompt, so relative phrases are computable from the input alone — it never needs a wall clock. The prompt format, byte for byte, is:

```
<reference ISO date> | <phrase> => <answer ISO date>
```

Seventeen surface renderers, a byte-level tokenizer (vocabulary 256, so every character is one token and there is no vocabulary file), and prompt-masked supervised fine-tuning in which only the ten-character ISO answer is scored — never the phrase the model was handed.

2. A model that looked perfect by cheating

The first trained model scored **100% on every absolute date form**, and I almost shipped it as a date normalizer. Then I typed an example by hand with a deliberate mismatch, and it failed instantly:

```
2024-03-10 | Jun 12 2023 => 2024-03-10 ← wrong: it echoed the reference date
```

The model was not parsing “Jun 12 2023” at all — it was **copying the reference date** from the front of the prompt. The cause was in the data generator: for absolute forms I had set the reference date *equal to* the answer, so during training the answer was always sitting right there at the start of the prompt, and the model learned the obvious shortcut. It scored 100% on a held-out set that shared the same leak. The fix is one line of intent: for absolute phrases, draw the reference date **independently** of the answer, so the only way to be right is to actually parse the phrase. That single change also lifted variable-count arithmetic from 32–45% to 77–81% — forcing real computation everywhere helped.

“Correct by construction” does not mean “not leaking the answer through a side channel.” Your evaluation will happily confirm a model that learned your bug; the only thing that caught this one was feeding it an input the generator could never produce.

3. Where a one-million-parameter model actually breaks

After the fix, Figure 1 is the honest capability map. Everything the model can do, it does reliably; the interesting part is the bottom row. Resolving a weekday phrase means mapping an arbitrary date to its day-of-week and then doing modular-7 arithmetic to find the next matching day. That day-of-week function is the most globally entangled computation in the whole task — it depends on the full date, including century and leap-year structure — and a 1M-parameter model simply does not have the capacity to represent it. Adding training did not move

the number: it is a capacity ceiling, not undertraining, and it is a clean, legible one.

Capability	Examples	Accuracy
Parse absolute dates	June 12, 2023 · the 12th of June 2023	100%
Resolve simple relatives	today · tomorrow · next/last week · next month · in N months	98-100%
Variable-N day/week math	in N days · N days ago · in N weeks	77-81%
Weekday resolution	next friday · last tuesday	~12%
Overall	mixed held-out set	85.4%

Table 1. The same numbers as Figure 1, with example phrases. The honest framing: nano-dates parses absolute dates and resolves most relative offsets; weekday arithmetic is where it stops. That is a far more useful and trustworthy thing to publish than “100% accurate date parser.”

4. Model and training at a glance

Parameters	1,016,960
Architecture	decoder-only transformer, pre-norm: RMSNorm, RoPE, grouped-query attention, SwiGLU
Tokenizer	raw UTF-8 bytes (vocabulary 256, no vocabulary file)
Width / depth / heads	dim 128 / 4 layers / 4 heads (2 KV heads), context 64 bytes
Training	prompt-masked SFT, 12k steps, batch 64, AdamW, cosine LR 3e-3
Data	100,000 code-generated pairs, 17 surface renderers
Final validation loss	0.036

Inference needs only `torch` and `safetensors` and a single self-contained model file. The full training, evaluation, and data-generation code — with tests that check the labels are correct and the answer is not leaked — reproduces the 85.4% number bit-for-bit and is in the public repository.

References

- [1] V. Rosić. *nano-dates* — model weights and self-contained inference. Hugging Face model repository, vukrosic/nano-dates, 2026.
- [2] V. Rosić. *nano-dates* — training, evaluation, and data-generation code. GitHub repository, vukrosic/nano-dates, 2026. MIT license.

***Author’s note.** This is an independent technical report by Vuk Rosić, June 2026, on his own open-source project. It is a method demonstration and a capability study, not a production date library — for real software, libraries such as `dateutil` and `chrono` are exact and free. The value here is showing how far a nano-scale model gets on a formal task trained entirely on synthetic data, and drawing a legible line at where its reasoning breaks. The figures are original to this report. Any interpretation errors are the author’s own.*